Чернівецький національний університет
імені Юрія Федьковича
Навчально-науковий інститут фізико-технічних
та комп'ютерних наук

ЗВІТ
про виконання лабораторної роботи №7
з дисципліни «Веб-програмування»

Тема: JavaScript-сценарії. Динамічні елементи

Виконав: студент групи 241-А
Гінчак Максим Богданович
Перевірив:
асистент Нестеренко Володимир Васильович

Чернівці · 2026

Завдання та варіант №5

Розробити динамічні елементи навчального сайту ByteMarket за допомогою JavaScript. За основу взято поточні сторінки лабораторної №6. Результати сценаріїв виводяться безпосередньо на сторінці, у текстові поля textarea та журнал подій.

Пункт методички	Реалізація
1. Спільне завдання	Обробники подій BODY на головній сторінці: завантаження, фокус, розмір вікна, друк, стан мережі, зміна якоря та вихід.
2. Спільне завдання	Наведення, клік, подвійний клік, натискання й відпускання кнопки миші на заголовках, b та i; зміна оформлення і повідомлення.
3.5. Варіант 5	Кнопка Run Task виводить доступні властивості об’єкта document.
4. Додаткове	Події миші для кількох зображень: насиченість, масштаб, рамка та повідомлення.
5.2.5. Варіант 5	Регулярними виразами знайти слова, які починаються літерою «п» і закінчуються літерою «я».
6.5. Додаткове, варіант 5	Кнопка Array Task створює масив B із масиву A без нульових елементів.

Файли та зміни порівняно з №6

• javascript.html - нова сторінка з трьома задачами та зображеннями.

• assets/js/tasks.js - властивості document, пошук та масиви.

• assets/js/events.js - події тексту й зображень, повідомлення та журнал.

• assets/css/javascript.css - оформлення нових блоків і станів елементів.

• index.html - обробники BODY; усі 9 сторінок отримали навігацію JavaScript і журнал.

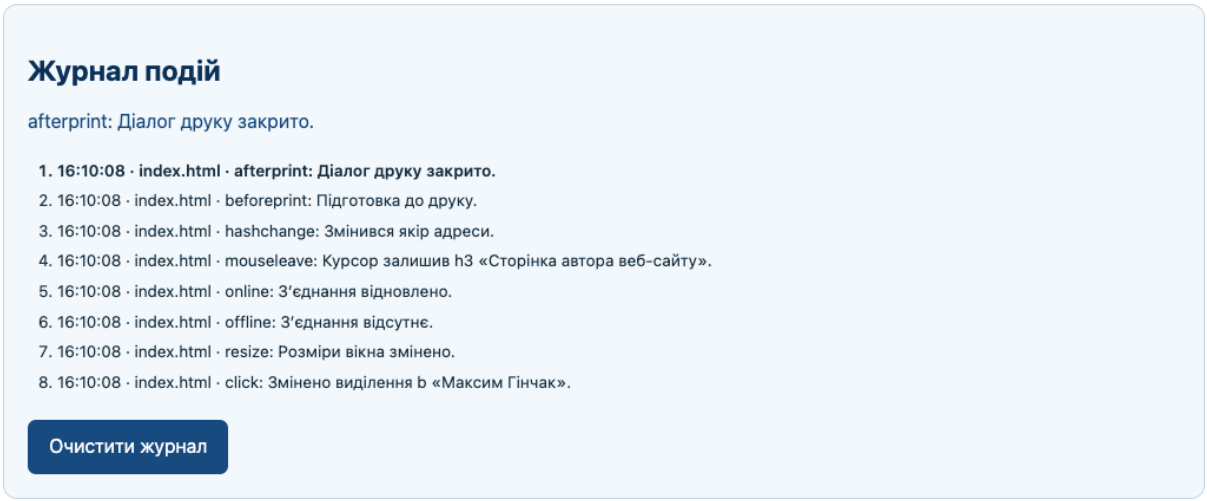
• Зміни_порівняно_з_лаб6.md та report/Захист.md - пояснення і порядок показу.

Використано зовнішні сценарії з defer: вони запускаються після розбору HTML. Власний локальний проєкт має Git-історію етапів роботи й окремий публічний репозиторій. Сценарії цієї роботи виконуються в браузері.

Приклад 1. Події вікна, тексту та зображень

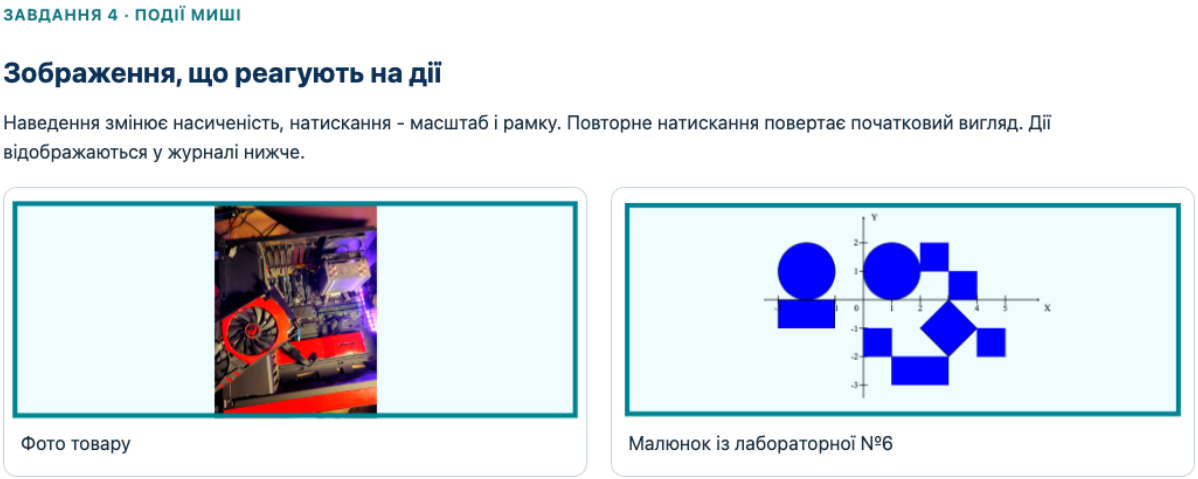
Сторінки: index.html і javascript.html. Реальні скриншоти у Chrome.

На головній сторінці обробники записано в атрибутах BODY. Повідомлення містять час, сторінку, назву події та опис. Останні 40 записів зберігаються в sessionStorage, тому перехід на іншу сторінку не стирає журнал.




```
<body
  onload="LabEvents.record('load', 'Завантажено.')"
  onresize="LabEvents.record('resize', 'Новий розмір.')"
  onbeforeprint="LabEvents.record('beforeprint', 'Друк.')">
...
</body>
```

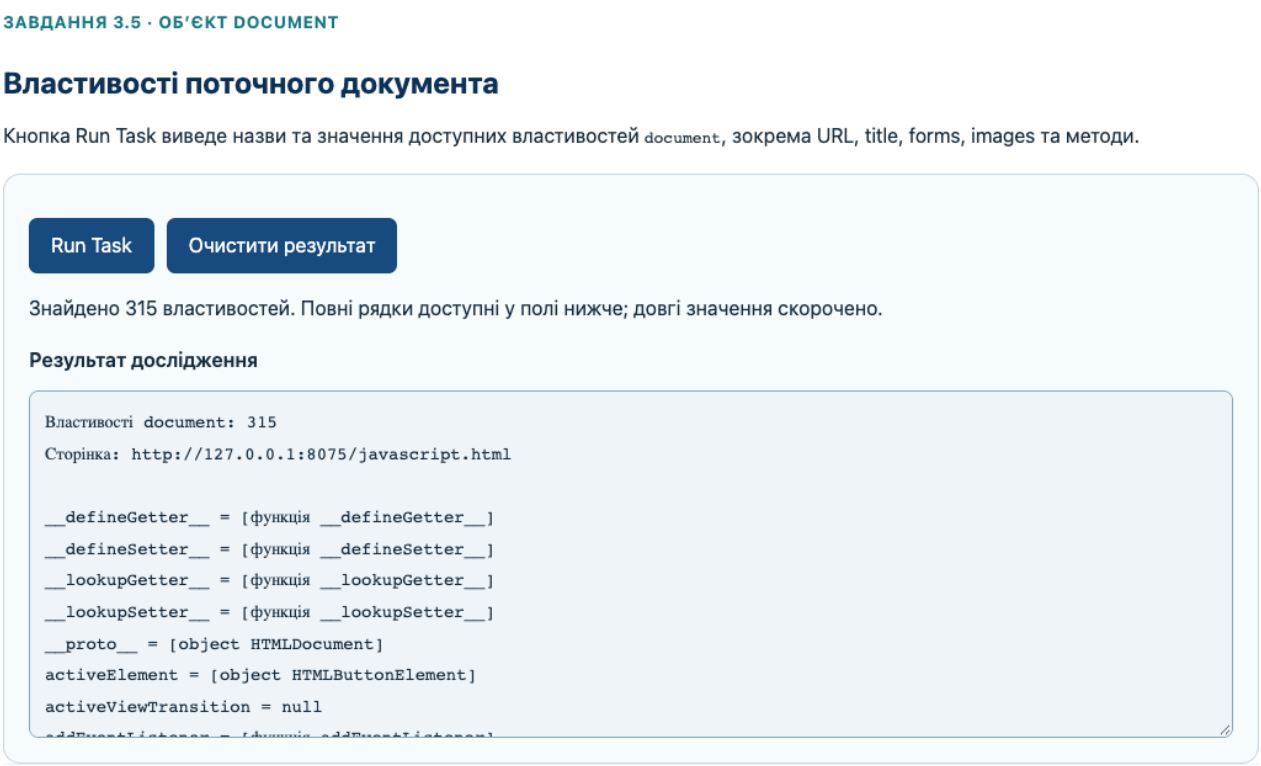
Наведення підсвічує заголовки й виділений текст; клік перемикає підкреслення, подвійний клік змінює колір і нахил. Для зображень клік змінює масштаб та рамку. Клавіші Enter і пробіл дублюють дію кліку.



Використано коректну назву hashchange. Для виходу зі сторінки - pagehide: браузер не гарантує unload. Перелік подій у методичці є набором прикладів; за умовою потрібно обробити декілька подій.

Приклад 2. Властивості об’єкта document

Сторінка: javascript.html. Скріншот реального сайту у Chrome, ширина вікна 1280 px.



У перевіреному Chrome знайдено 315 властивостей. Кількість залежить від браузера. Виводяться назви й короткі представлення значень: URL, title, forms, images, методи та успадковані властивості. Довгі рядки скорочуються до 160 символів, функції й об’єкти показуються описово.

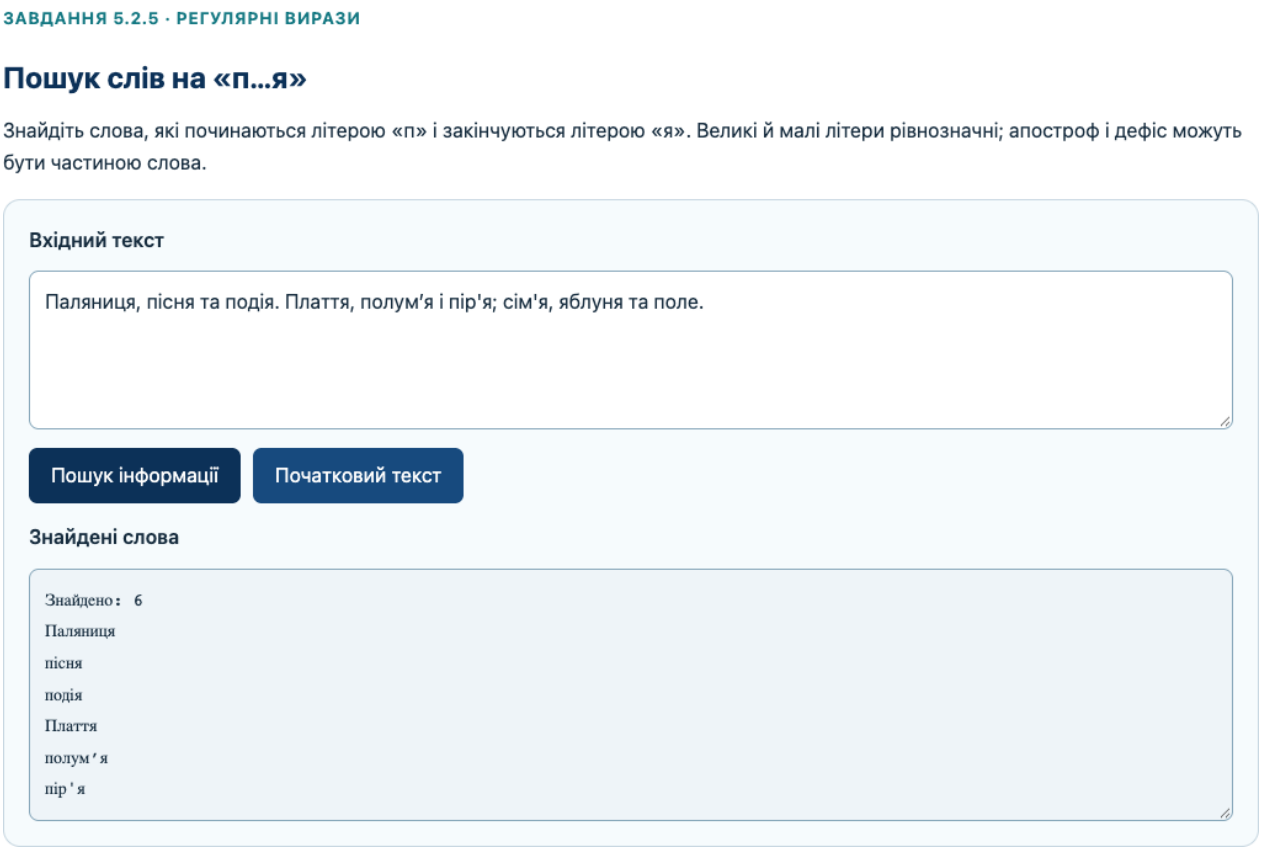
```
const names = new Set();
for (const name in document) names.add(name);
for (let object = document; object !== null;
     object = Object.getPrototypeOf(object)) {
  for (const name of Reflect.ownKeys(object)) {
    names.add(name);
  }
}
```

Цикл for...in збирає перелічувані властивості. Reflect.ownKeys доповнює їх неперелічуваними і символьними ключами на кожному рівні прототипів. Set прибирає повтори. Значення читаються через Reflect.get у try...catch: недоступна властивість не перериває весь сценарій.

Обробник submit викликає preventDefault, щоб браузер не перезавантажував сторінку. Результат записується у value поля textarea. document.write не використано, оскільки після завантаження він може замінити поточний документ.

Приклад 3. Слова на «п…я»

Сторінка: javascript.html. Скріншот реального сайту у Chrome, ширина вікна 1280 px.



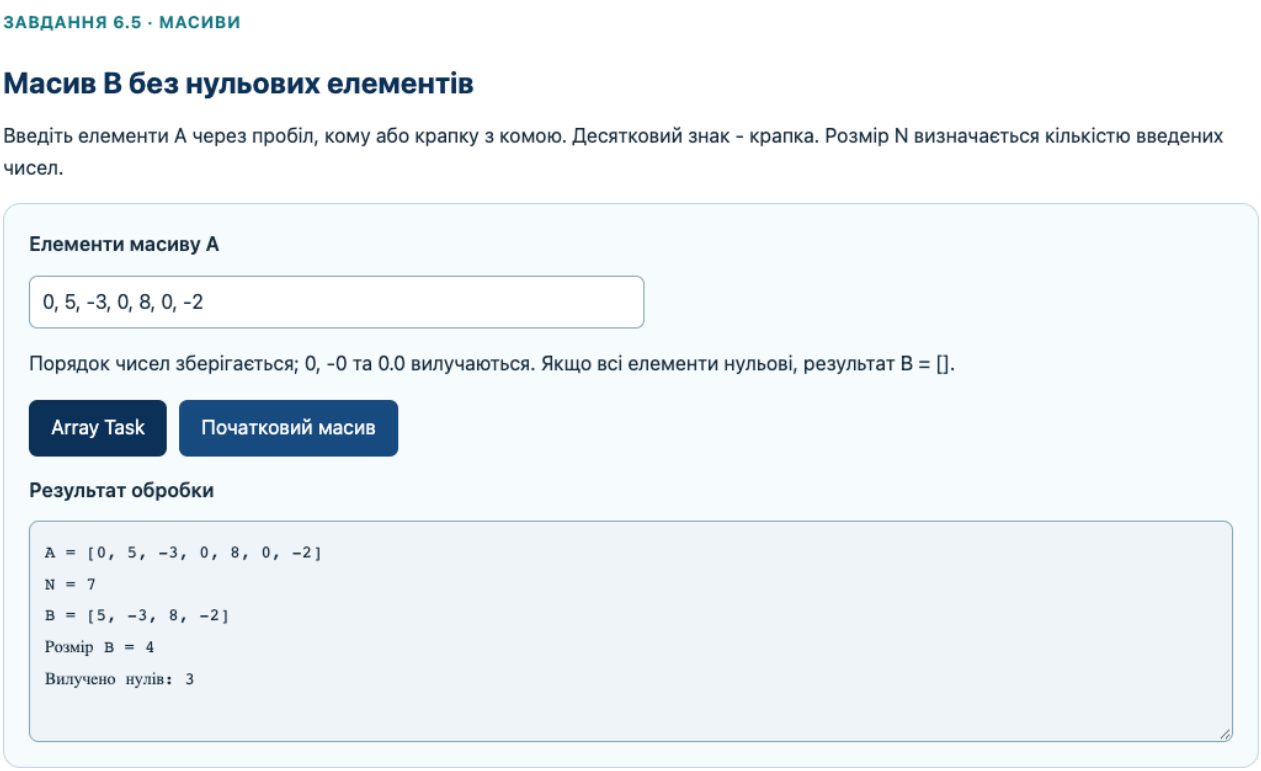

```
const words = text.match(
  /\p{L}[\p{L}\p{M}]*(?:['’-]\p{L}[\p{L}\p{M}]*)*/gu
) || [];
const matches = words.filter(word => /^п[\s\S]*я$/iu.test(word));
```

Перший вираз виділяє слова: літери Unicode, комбіновані знаки, внутрішній апостроф або дефіс. Другий перевіряє ціле слово: ^ - початок, п - перша літера, * - довільна кількість символів, я - остання літера, $ - кінець. Прапорець i вимикає залежність від регістру, u додає підтримку Unicode, g збирає всі слова.

У прикладі знайдено 6 слів: Паляниця, пісня, подія, Плаття, полум’я, пір'я. Слова сім'я, яблуня та поле відкидаються. Повтори зберігаються, оскільки шукаються всі входження. Якщо збігів немає, виводиться кількість 0 і пояснення.

Приклад 4. Масив без нульових елементів

Сторінка: javascript.html. Скріншот реального сайту у Chrome, ширина вікна 1280 px.




```
const tokens = field.value.trim().split(/[\s,;]+/u).filter(Boolean);
const a = tokens.map(Number);
const b = a.filter(value => value !== 0);
```

Перед перетворенням перевіряються формат кожного числа та Number.isFinite. Підтримуються від’ємні числа, десяткові дроби з крапкою та експоненційний запис. Некоректний текст позначається через setCustomValidity і reportValidity. Фрагмент вище показує основний алгоритм після перевірки введення.

Для A = [0, 5, -3, 0, 8, 0, -2] отримано B = [5, -3, 8, -2]. N = 7, розмір B = 4, вилучено 3 нулі. Метод filter створює новий масив і зберігає порядок решти елементів. Записи -0 та 0.0 також дорівнюють нулю. Для масиву з одних нулів B буде порожнім: [].

Основні елементи форми

form об’єднує поля; label пов’язує підпис із полем; input приймає числа як текст; textarea містить вхідний текст або результат; button запускає дію. Атрибут readonly дозволяє копіювати результат і забороняє змінювати його. Кнопки вимкнені, доки сценарій не завантажиться; без JavaScript показано пояснення.

Тестові приклади та перевірка

Перевірка	Очікуваний і отриманий результат
Run Task	315 властивостей; наявні URL, title, forms, images, getElementById та символьні ключі.
Пошук у початковому тексті	6 слів: Паляниця, пісня, подія, Плаття, полум’я, пір'я.
Регістр, апостроф, відсутність збігів	Великі літери та три форми апострофа обробляються; за відсутності слів кількість дорівнює 0.
A: 0, 5, -3, 0, 8, 0, -2	B: 5, -3, 8, -2; вилучено 3 нулі.
A: 0 -0 0.0 +0	B = []; розмір 0.
A: -2, 3.5; 0 4e1 0.01	B = [-2, 3.5, 40, 0.01].
Некоректні числа та порожнє поле	abc, Infinity, 1e999, 0x10, 1..2 і порожнє введення відхилено.
Події та клавіатура	Завантаження, перехід, розмір, мережа, якір і друк заносяться в журнал; текст і зображення змінюють стиль.
390 px; JavaScript / сховище вимкнені	Нова сторінка без горизонтального виходу; без JS дії вимкнені; блокування сховища не ламає задачі.
Навігація і локальні файли	Перевірено 9 HTML-сторінок та 142 локальних посилань. Помилок виконання у браузері немає.

Перевірку виконано у Chrome за допомогою Playwright. Число властивостей document та їхні значення залежать від браузера й поточного стану сторінки. Детальні результати збережено у report/check-results.json.

Висновки

Розроблено динамічні елементи сайту засобами JavaScript: обробку подій, перегляд властивостей документа, пошук слів регулярними виразами та фільтрування масиву. Для варіанта №5 реалізовано пункти 3.5, 5.2.5 і 6.5, а також спільні завдання й обробку зображень. Порівняно з лабораторною №6 додано інтерактивну сторінку та сценарії роботи з DOM і даними.

Репозиторій

https://github.com/MaksHinchak/html-lab-07-variant-5